Chapter 14 JavaFX Basics

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Motivations

- JavaFX is a new framework for developing Java GUI programs.
- Graphical User Interface (GUI)
  - provides user-friendly human interaction
- The JavaFX API (Application Programming Interface) is an excellent example of how the object-oriented principle is applied.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Motivations

- This chapter serves two purposes.
  - First, it presents the basics of JavaFX programming.
  - Second, it uses JavaFX to demonstrate OOP.
- Specifically, this chapter introduces the framework of JavaFX and discusses JavaFX GUI components and their relationships.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Objectives

- To distinguish between JavaFX, Swing, and AWT (§14.2).
- To write a simple JavaFX program and understand the relationship among stages, scenes, and nodes (§14.3).
- To create user interfaces using panes, UI controls, and shapes (§14.4).
- To use binding properties to synchronize property values (§14.5).
- To use the common properties **style** and **rotate** for nodes (§14.6).
- To create colors using the **Color** class (§14.7).
- To create fonts using the **Font** class (§14.8).
- To create images using the **Image** class and to create image views using the **ImageView** class (§14.9).
- To layout nodes using **Pane**, **StackPane**, **FlowPane**, **GridPane**, **BorderPane**, **HBox**, and **VBox** (§14.10).
- To display text using the **Text** class and create shapes using **Line**, **Circle**, **Rectangle**, **Ellipse**, **Arc**, **Polygon**, and **Polyline** (§14.11).
- To develop the reusable GUI components **ClockPane** for displaying an analog clock (§14.12).

Copyright © 2022 Pearson Education Ltd. All rights reserved.

JavaFX vs Swing and AWT

- Swing and AWT are replaced by the JavaFX platform for developing rich Internet applications.

- When Java was introduced, the GUI classes were bundled in a library known as the *Abstract Windows Toolkit (AWT)*.
  - AWT is fine for developing simple graphical user interfaces, but not for developing comprehensive GUI projects.
  - In addition, AWT is prone to platform-specific bugs.

- The AWT user-interface components were replaced by a more robust, versatile, and flexible library known as *Swing components*.
  - Swing components are painted directly on canvases using Java code.
  - Swing components depend less on the target platform and use less of the native GUI resource.

- With the release of Java 8, Swing is replaced by a completely new GUI platform known as *JavaFX*.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Basic Structure of JavaFX

- Application
  - *The abstract* **javafx.application.Application** *class defines the essential framework for writing JavaFX programs.*

```
public class MyJavaFX extends Application
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Basic Structure of JavaFX

- The **launch** method is a static method defined in the **Application** class for launching a stand-alone JavaFX application.
- The **main** method is not needed if you run the program from the command line.
- It may be needed to launch a JavaFX program from an IDE with a limited JavaFX support.
- When you run a JavaFX application without a main method, JVM automatically invokes the **launch** method to run the application.

```
public static void main(String[] args) {
      Application.launch(args);
 }
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Basic Structure of JavaFX

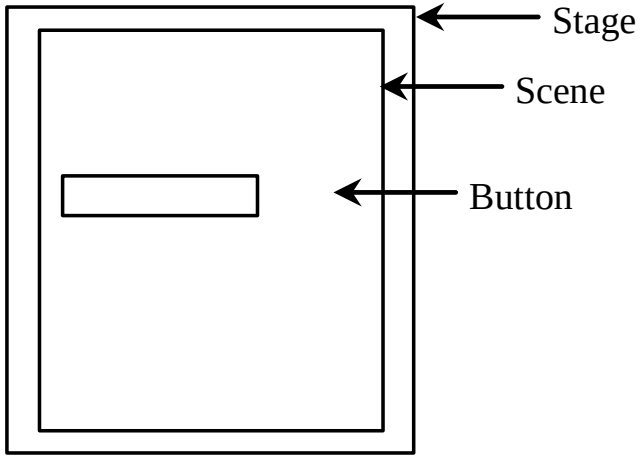
- The main class overrides the **start** method defined in **javafx.application.Application.**
- After a JavaFX application is launched, the JVM constructs an instance of the class using its **no-arg** constructor and invokes its **start** method.
- The **start** method normally places UI controls in a scene and displays the scene in a stage.

```
@Override // Override the start method in the Application class
public void start(Stage primaryStage) {

        …

}
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Basic Structure of JavaFX

- Application
- Override the start(Stage) method
- Stage, Scene, and Nodes

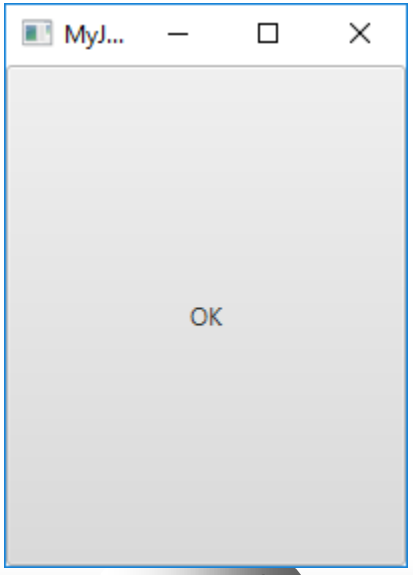


MyJavaFX

MultipleStageDemo

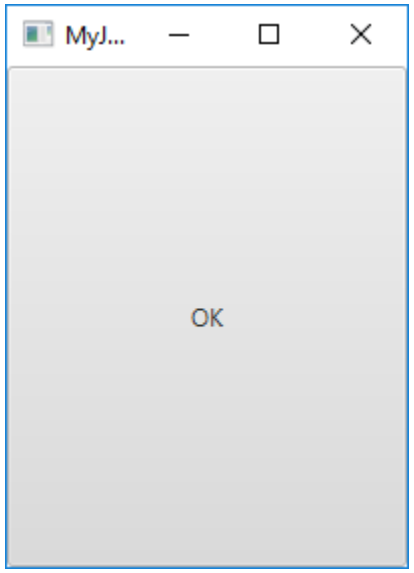
Copyright © 2022 Pearson Education Ltd. All rights reserved.

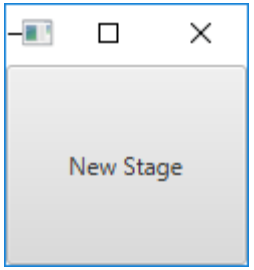
```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;
public class MyJavaFX extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a button and place it in the scene
    Button btOK = new Button("OK");
    Scene scene = new Scene(btOK, 200, 250);
    primaryStage.setTitle("MyJavaFX"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
/**
* The main method is only needed for the IDE with limited
* JavaFX support. Not needed for running from the command line.
*/
  public static void main(String[] args) {
     launch(args);
  }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;

public class MultipleStageDemo extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a scene and place a button in the scene
    Scene scene = new Scene(new Button("OK"), 200, 250);
    primaryStage.setTitle("MyJavaFX"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage

    Stage stage = new Stage(); // Create a new stage
    stage.setTitle("Second Stage"); // Set the stage title
    // Set a scene with a button in the stage
    stage.setScene(new Scene(new Button("New Stage"), 100, 100));
    stage.show(); // Display the stage
  }

  /**
   * The main method is only needed for the IDE with limited
   * JavaFX support. Not needed for running from the command line.
   */
  public static void main(String[] args) {
    launch(args);
  }
}
```





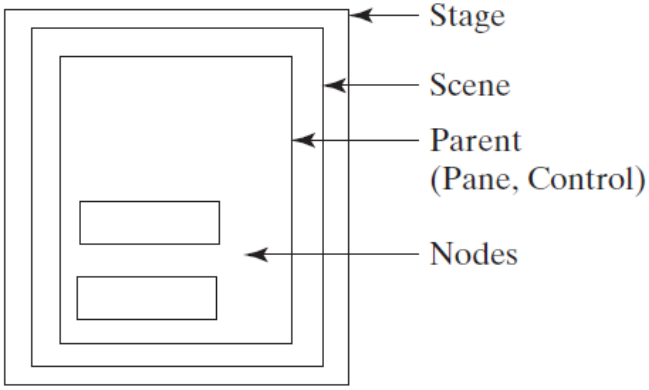
Copyright © 2022 Pearson Education Ltd. All rights reserved.

Panes, UI Controls, and Shapes

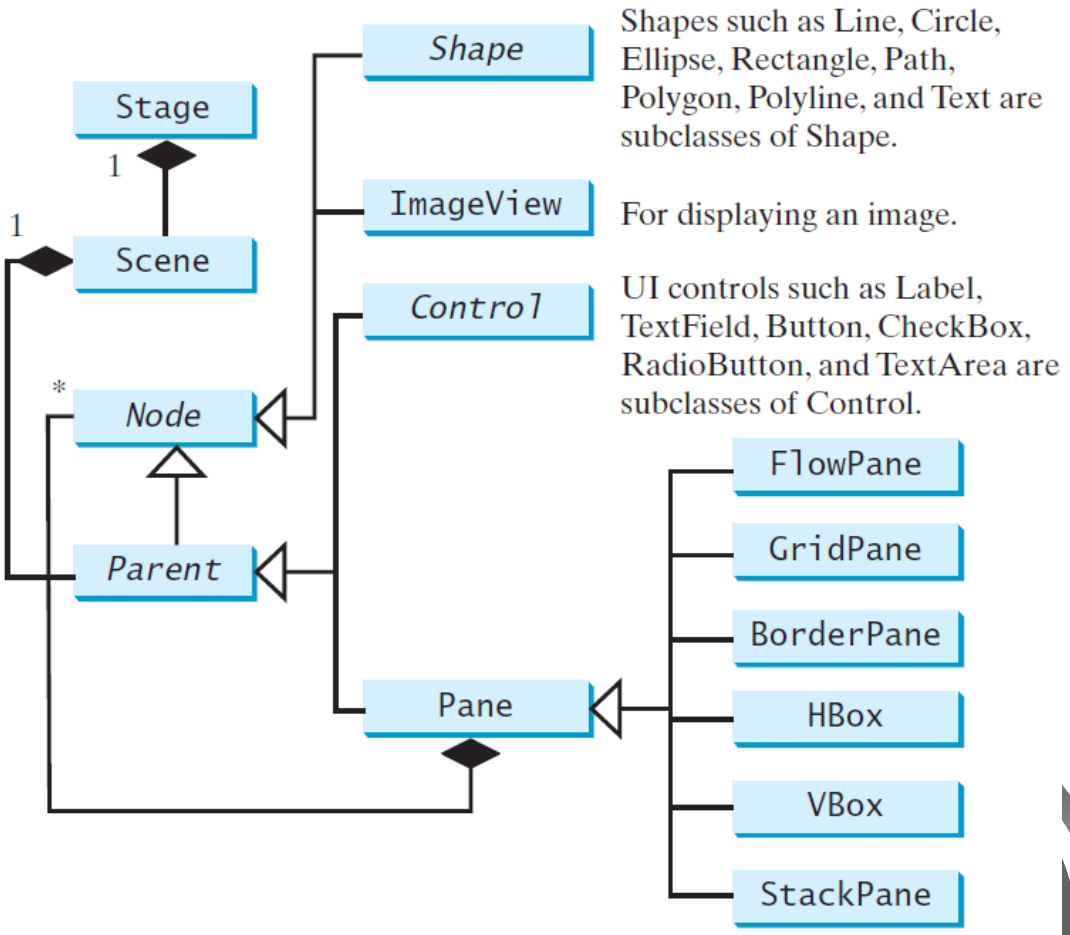
- *Panes, UI controls, and shapes are subtypes of* **Node**.
- A *node* is a visual component such as a shape, an image view, a UI control, or a pane.
- A *shape* refers to a text, line, circle, ellipse, rectangle, arc, polygon, polyline, etc.
- A *UI control* refers to a label, button, check box, radio button, text field, text area, etc.
- *Panes* are container classes for automatically laying out the nodes in a desired location and size.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Panes, UI Controls, and Shapes



(a)

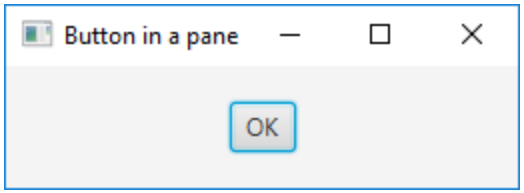


(b)

ButtonInPane

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Button;

public class ButtonInPane extends Application {

  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a scene and place a button in the scene
    StackPane pane = new StackPane();
    pane.getChildren().add(new Button("OK"));
    Scene scene = new Scene(pane, 200, 50);
    primaryStage.setTitle("Button in a pane"); // Set the stage    title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }

  public static void main(String[] args) {
    launch(args);
  }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

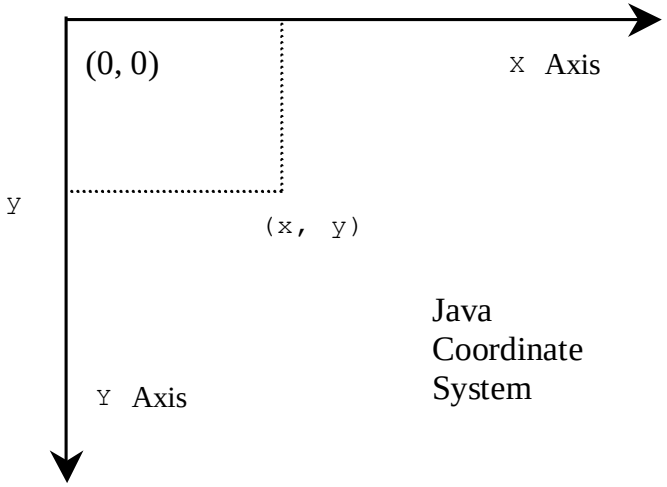
Panes

- The program creates a **StackPane** and adds a button as a child of the pane.
- The **getChildren()** method returns an instance of **javafx.collections.ObservableList**.
- **ObservableList** behaves very much like an **ArrayList** for storing a collection of elements.
- Invoking **add(e)** adds an element to the list.
- The **StackPane** places the nodes in the center of the pane on top of each other.

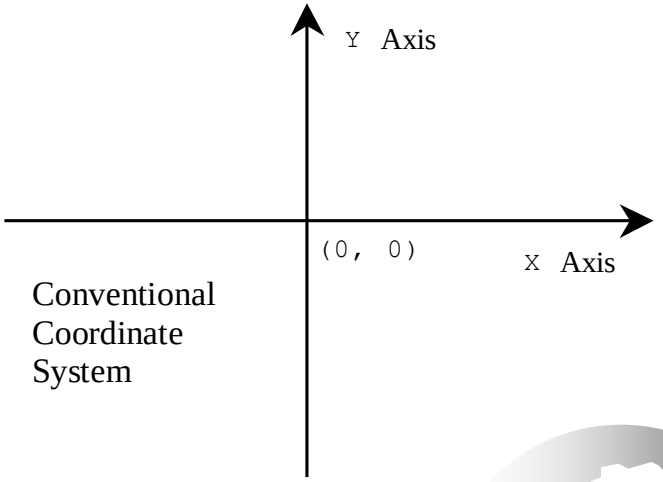
Copyright © 2022 Pearson Education Ltd. All rights reserved.

Display a Shape

This example displays a circle in the center of the pane.



Java Coordinate System

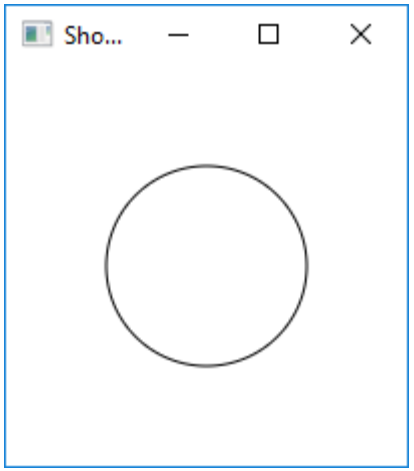


Conventional Coordinate System

ShowCircle

Copyright © 2022 Pearson Education Ltd. All rights reserved.

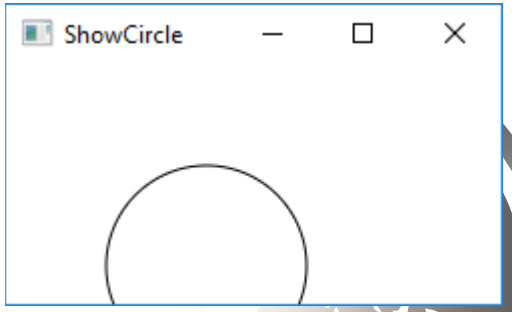
```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.shape.Circle;
import javafx.scene.paint.Color;

public class ShowCircle extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a circle and set its properties
    Circle circle = new Circle();
    circle.setCenterX(100);
    circle.setCenterY(100);
    circle.setRadius(50);
    circle.setStroke(Color.BLACK);
    circle.setFill(Color.WHITE);
    // Create a pane to hold the circle
    Pane pane = new Pane();
    pane.getChildren().add(circle);
    // Create a scene and place it in the stage
    Scene scene = new Scene(pane, 200, 200);
    primaryStage.setTitle("ShowCircle"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
  /**
  * The main method is only needed for the IDE with limited
  * JavaFX support. Not needed for running from the command line.
  */
  public static void main(String[] args) {
    launch(args);
  }

}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Binding Properties

- JavaFX introduces a new concept called ***binding property*** that enables a *target object* to be bound to a *source object*.
  - If the value in the source object changes, the target property is also changed automatically.
  - The target object is simply called a *binding object* or a *binding property*.
  - The source object is called a *bindable object* or *observable object*.
- Resizing the window in the previous example would cover the object:

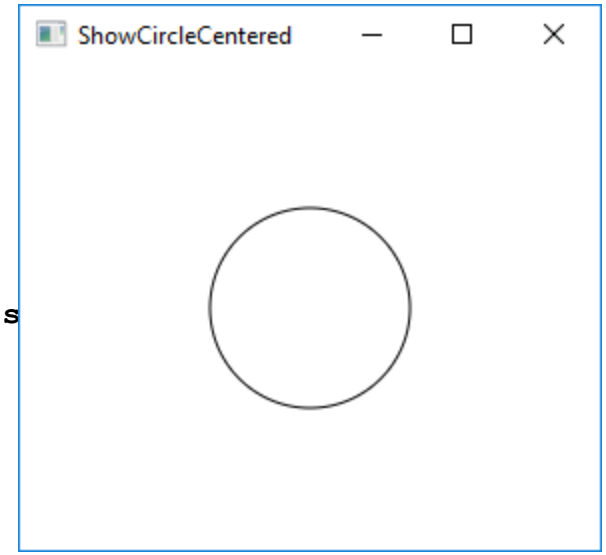


- Example: `target.bind(source);`

ShowCircleCentered

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.shape.Circle;
import javafx.scene.paint.Color;
public class ShowCircleCentered extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a pane to hold the circle
    Pane pane = new Pane();

    // Create a circle and set its properties
    Circle circle = new Circle();
    circle.centerXProperty().bind(pane.widthProperty().divide(2));
    circle.centerYProperty().bind(pane.heightProperty().divide(2));
    circle.setRadius(50);
    circle.setStroke(Color.BLACK);
    circle.setFill(Color.WHITE);
    pane.getChildren().add(circle); // Add circle to the pane

    // Create a scene and place it in the stage
    Scene scene = new Scene(pane, 200, 200);
    primaryStage.setTitle("ShowCircleCentered"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
  /**
  * The main method is only needed for the IDE with limited
  * JavaFX support. Not needed for running from the command line.
  */
  public static void main(String[] args) {
    launch(args);
  }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Binding Property: getter, setter, and property getter

```java
public class SomeClassName {

  private PropertyType x;

  /** Value getter method */
  public propertyValueType getX() { ... }

  /** Value setter method */
  public void setX(propertyValueType value) { ... }

  /** Property getter method */
  public PropertyType
    xProperty() { ... }
}
```

(a) x is a binding property

```java
public class Circle {

  private DoubleProperty centerX;

  /** Value getter method */
  public double getCenterX() { ... }

  /** Value setter method */
  public void setCenterX(double value) { ... }

  /** Property getter method */
  public DoubleProperty centerXProperty() { ... }
}
```

(b) centerX is binding property

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Unidirectional Binding

LISTING 14.6 BindingDemo.java

```java
import javafx.beans.property.DoubleProperty;
import javafx.beans.property.SimpleDoubleProperty;

public class BindingDemo {
  public static void main(String[] args) {
    DoubleProperty d1 = new SimpleDoubleProperty(1);
    DoubleProperty d2 = new SimpleDoubleProperty(2);
    d1.bind(d2);
    System.out.println("d1 is " + d1.getValue()
      + " and d2 is " + d2.getValue());
    d2.setValue(70.2);
    System.out.println("d1 is " + d1.getValue()
      + " and d2 is " + d2.getValue());
  }
}
```

```
d1 is 2.0 and d2 is 2.0
d1 is 70.2 and d2 is 2.0
```

- *Bidirectional binding:* If the target and source are both binding properties and observable properties, they can be bound bidirectionally using the **bindBidirectional** method.

  See **BidirectionalBindingDemo.java**

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Uni/Bidirectional Binding

BindingDemo

BidirectionalBindingDemo

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Common Properties and Methods for Nodes

- *The abstract* **Node** *class defines many properties and methods that are common to all nodes.*
- *Two common properties:*
  - *style:* set a JavaFX CSS style
  - *rotate:* Rotate a node

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Common Properties and Methods for Nodes

- JavaFX style properties are similar to cascading style sheets (CSS) used to specify the styles for HTML elements in a Web page.
- So, the style properties in JavaFX are called *JavaFX CSS*.
- In JavaFX, a style property is defined with a prefix **-fx-**
- Each node has its own style properties.
  - You can find these properties from http://docs.oracle.com/javafx/2/api/javafx/scene/doc-files/cssref.html

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Common Properties and Methods for Nodes

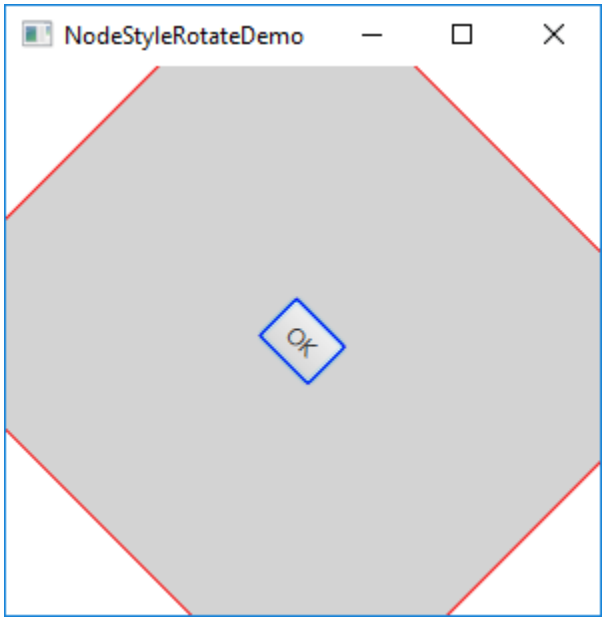
- The syntax for setting a style is **styleName:value**.
- Multiple style properties for a node can be set together separated by semicolon (**;**).
- For example, the following statement

```
circle.setStyle("-fx-stroke: black; -fx-fill:
red;");
```

  sets two JavaFX CSS properties for a circle.
- rotate: Rotate a node

NodeStyleRotateDemo

Copyright © 2022 Pearson Education Ltd. All rights reserved.




```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Button;
public class NodeStyleRotateDemo extends Application {
  @Override
  public void start(Stage primaryStage) {
    StackPane pane = new StackPane();
    Button btOK = new Button("OK");
    btOK.setStyle("-fx-border-color: blue;");
    pane.getChildren().add(btOK);
    pane.setRotate(45);
    pane.setStyle("-fx-border-color: red; -fx-background-color: lightgray;");
    Scene scene = new Scene(pane, 200, 250);
    primaryStage.setTitle("NodeStyleRotateDemo"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
  /**
  * The main method is only needed for the IDE with limited
  * JavaFX support. Not needed for running from the command line.
  */
  public static void main(String[] args) {
    launch(args);
  }

}
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

```
public class NodeStyleRotateDemo extends Application {
  @Override
  public void start(Stage primaryStage) {
    StackPane pane = new StackPane();
  ...
}
```

```
/* The StackPane layout pane places all of the nodes within
a single stack with each new node added on top of the
previous node. This layout model provides an easy way to
overlay text on a shape or image and to overlap common
shapes to create a complex shape. */
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

The Color Class

The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.paint.Color

-red: double	The red value of this Color (between 0.0 and 1.0).
-green: double	The green value of this Color (between 0.0 and 1.0).
-blue: double	The blue value of this Color (between 0.0 and 1.0).
-opacity: double	The opacity of this Color (between 0.0 and 1.0).
+Color(r: double, g: double, b: double, opacity: double)	Creates a Color with the specified red, green, blue, and opacity values.
+brighter(): Color	Creates a Color that is a brighter version of this Color.
+darker(): Color	Creates a Color that is a darker version of this Color.
+color(r: double, g: double, b: double): Color	Creates an opaque Color with the specified red, green, and blue values.
+color(r: double, g: double, b: double, opacity: double): Color	Creates a Color with the specified red, green, blue, and opacity values.
+rgb(r: int, g: int, b: int): Color	Creates a Color with the specified red, green, and blue values in the range from 0 to 255.
+rgb(r: int, g: int, b: int, opacity: double): Color	Creates a Color with the specified red, green, and blue values in the range from 0 to 255 and a given opacity.

https://liveexample.pearsoncmg.com/dsanimation/FigureSection14_7.html

Copyright © 2022 Pearson Education Ltd. All rights reserved.

The Font Class

The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

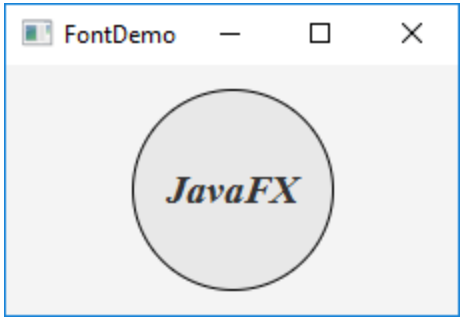
javafx.scene.text.Font

-size: double	The size of this font.
-name: String	The name of this font.
-family: String	The family of this font.
+Font(size: double)	Creates a Font with the specified size.
+Font(name: String, size: double)	Creates a Font with the specified full font name and size.
+font(name: String, size: double)	Creates a Font with the specified name and size.
+font(name: String, w: FontWeight, size: double)	Creates a Font with the specified name, weight, and size.
+font(name: String, w: FontWeight, p: FontPosture, size: double)	Creates a Font with the specified name, weight, posture, and size.
+getFamilies(): List<String>	Returns a list of font family names.
+getFontNames(): List<String>	Returns a list of full font names including family and weight.

FontDemo

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
public class FontDemo extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a pane to hold the circle
    Pane pane = new StackPane();

    // Create a circle and set its properties
    Circle circle = new Circle();
    circle.setRadius(50);
    circle.setStroke(Color.BLACK);
    circle.setFill(new Color(0.5, 0.5, 0.5, 0.1));
    pane.getChildren().add(circle); // Add circle to the pane

    // Create a label and set its properties
    Label label = new Label("JavaFX");
    label.setFont(Font.font("Times New Roman", FontWeight.BOLD, FontPosture.ITALIC, 20));
    pane.getChildren().add(label);

    // Create a scene and place it in the stage
    Scene scene = new Scene(pane);
    primaryStage.setTitle("FontDemo"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }

  /**
   * The main method is only needed for the IDE with limited
   * JavaFX support. Not needed for running from the command line.
   */
  public static void main(String[] args) {
    launch(args);
  }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

The Image Class

The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.image.Image

```
-error: ReadOnlyBooleanProperty
-height: ReadOnlyBooleanProperty
-width: ReadOnlyBooleanProperty
-progress: ReadOnlyBooleanProperty
```

```
+Image(filenameOrURL: String)
```

Indicates whether the image is loaded correctly?

The height of the image.

The width of the image.

The approximate percentage of image's loading that is completed.

Creates an `Image` with contents loaded from a file or a URL.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

The ImageView Class

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.image.ImageView

```
-fitHeight: DoubleProperty
-fitWidth: DoubleProperty
-x: DoubleProperty
-y: DoubleProperty
-image: ObjectProperty<Image>
```

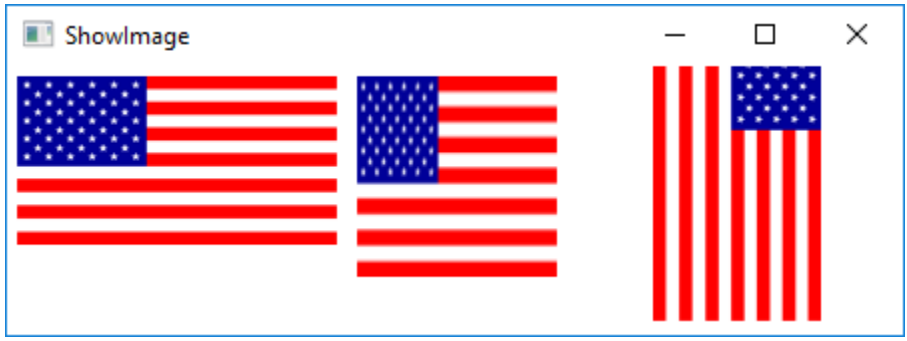
```
+ImageView()
+ImageView(image: Image)
+ImageView(filenameOrURL: String)
```

-fitHeight: DoubleProperty	The height of the bounding box within which the image is resized to fit.
-fitWidth: DoubleProperty	The width of the bounding box within which the image is resized to fit.
-x: DoubleProperty	The x-coordinate of the ImageView origin.
-y: DoubleProperty	The y-coordinate of the ImageView origin.
-image: ObjectProperty<Image>	The image to be displayed in the image view.
+ImageView()	Creates an `ImageView`.
+ImageView(image: Image)	Creates an `ImageView` with the specified image.
+ImageView(filenameOrURL: String)	Creates an `ImageView` with image loaded from the specified file or URL.

ShowImage

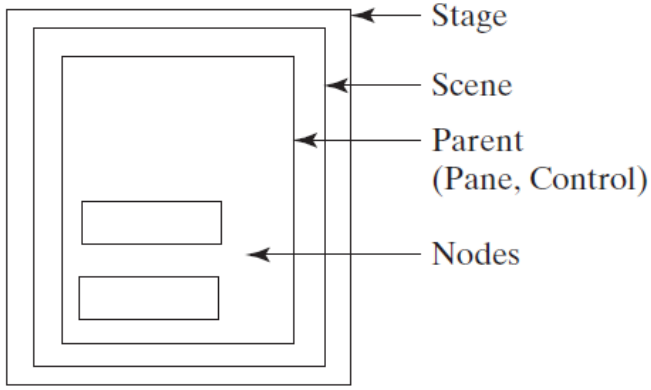
Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.layout.HBox;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.geometry.Insets;
public class ShowImage extends Application {
  @Override
  public void start(Stage primaryStage) {
    // Create a pane to hold the image views
    Pane pane = new HBox(10);
    pane.setPadding(new Insets(5, 5, 5, 5));
    Image image = new Image("image/us.gif");
    pane.getChildren().add(new ImageView(image));
    ImageView imageView2 = new ImageView(image);
    imageView2.setFitHeight(100);
    imageView2.setFitWidth(100);
    pane.getChildren().add(imageView2);
    ImageView imageView3 = new ImageView(image);
    imageView3.setRotate(90);
    pane.getChildren().add(imageView3);
    Scene scene = new Scene(pane);
    primaryStage.setTitle("ShowImage");
    primaryStage.setScene(scene);
    primaryStage.show();
  }
  public static void main(String[] args) {
    launch(args);
  }
}
```

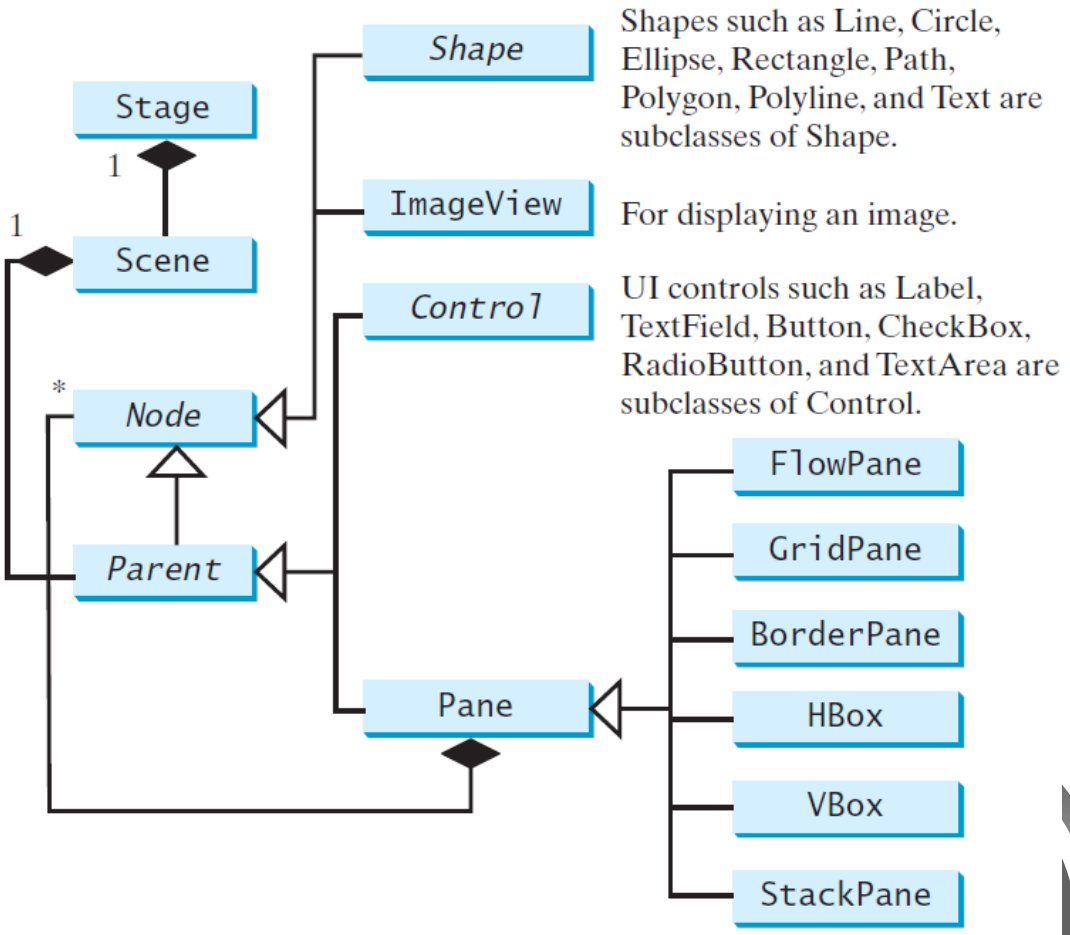


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Panes, UI Controls, and Shapes



(a)



(b)

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Layout Panes

JavaFX provides many types of panes for organizing nodes in a container.

Class	*Description*
`Pane`	Base class for layout panes. It contains the `getChildren()` method for returning a list of nodes in the pane.
`StackPane`	Places the nodes on top of each other in the center of the pane.
`FlowPane`	Places the nodes row-by-row horizontally or column-by-column vertically.
`GridPane`	Places the nodes in the cells in a two-dimensional grid.
`BorderPane`	Places the nodes in the top, right, bottom, left, and center regions.
`HBox`	Places the nodes in a single row.
`VBox`	Places the nodes in a single column.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

FlowPane

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.layout.FlowPane

```
-alignment: ObjectProperty<Pos>
-orientation:
   ObjectProperty<Orientation>
-hgap: DoubleProperty
-vgap: DoubleProperty
```

```
+FlowPane()
+FlowPane(hgap: double, vgap:
   double)
+FlowPane(orientation:
   ObjectProperty<Orientation>)
+FlowPane(orientation:
   ObjectProperty<Orientation>,
   hgap: double, vgap: double
```

The overall alignment of the content in this pane (default: `Pos.LEFT`).

The orientation in this pane (default: `Orientation.HORIZONTAL`).

The horizontal gap between the nodes (default: `0`).

The vertical gap between the nodes (default: `0`).

Creates a default `FlowPane`.

Creates a `FlowPane` with a specified horizontal and vertical gap.

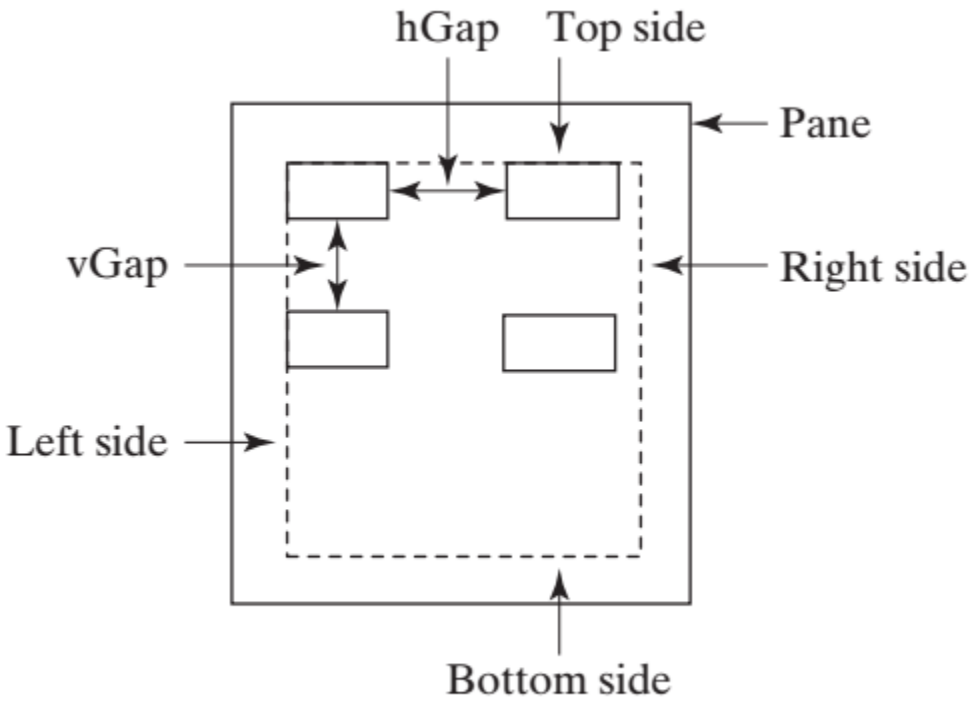
Creates a `FlowPane` with a specified orientation.

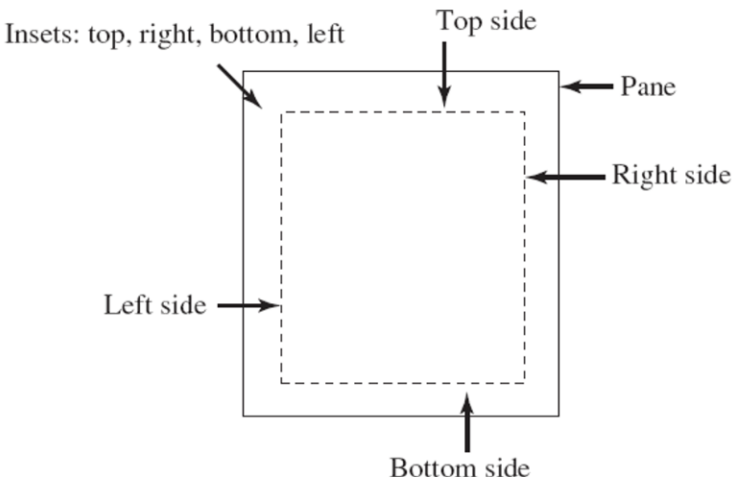
Creates a `FlowPane` with a specified orientation, horizontal gap and vertical gap.

MultipleStageDemo

Copyright © 2022 Pearson Education Ltd. All rights reserved.

FlowPane

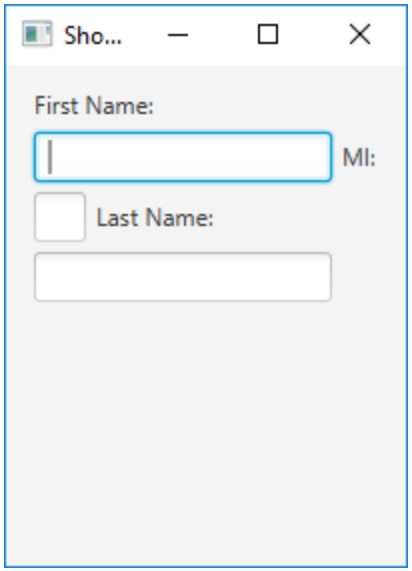


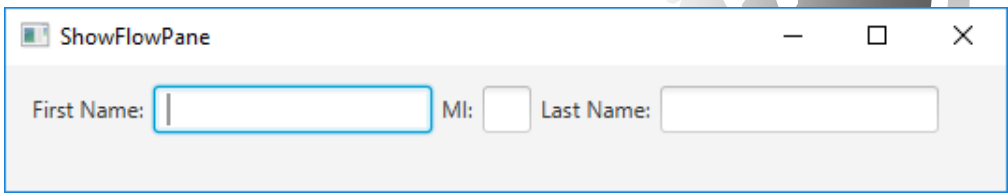


You can specify **hGap** and **vGap** between the nodes in a **FlowLPane**.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.FlowPane;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.geometry.Insets;
public class ShowFlowPane extends Application {
  @Override
  public void start(Stage primaryStage) {
    FlowPane pane = new FlowPane();
    pane.setPadding(new Insets(11, 12, 13, 14));
    pane.setHgap(5);
    pane.setVgap(5);
    // Place nodes in the pane
    pane.getChildren().addAll(new Label("First Name:"),
    new TextField(), new Label("MI:"));
    TextField tfMi = new TextField();
    tfMi.setPrefColumnCount(1);
    pane.getChildren().addAll(tfMi, new Label("Last Name:"),
      new TextField());
    // Create a scene and place it in the stage
    Scene scene = new Scene(pane, 210, 150);
    primaryStage.setTitle("ShowFlowPane");
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
  public static void main(String[] args)
    launch(args);
  }

}
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

GridPane

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

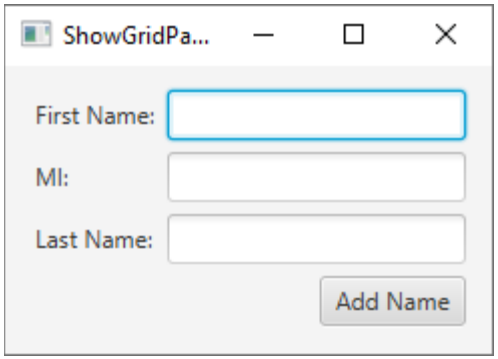
javafx.scene.layout.GridPane

-alignment: ObjectProperty<Pos>	The overall alignment of the content in this pane (default: `Pos.LEFT`).
-gridLinesVisible: BooleanProperty	Is the grid line visible? (default: `false`)
-hgap: DoubleProperty	The horizontal gap between the nodes (default: 0).
-vgap: DoubleProperty	The vertical gap between the nodes (default: 0).
+GridPane()	Creates a `GridPane`.
+add(child: Node, columnIndex: int, rowIndex: int): void	Adds a node to the specified column and row.
+addColumn(columnIndex: int, children: Node...): void	Adds multiple nodes to the specified column.
+addRow(rowIndex: int, children: Node...): void	Adds multiple nodes to the specified row.
+getColumnIndex(child: Node): int	Returns the column index for the specified node.
+setColumnIndex(child: Node, columnIndex: int): void	Sets a node to a new column. This method repositions the node.
+getRowIndex(child:Node): int	Returns the row index for the specified node.
+setRowIndex(child: Node, rowIndex: int): void	Sets a node to a new row. This method repositions the node.
+setHalighnment(child: Node, value: HPos): void	Sets the horizontal alignment for the child in the cell.
+setValighnment(child: Node, value: VPos): void	Sets the vertical alignment for the child in the cell.

ShowGridPane

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.GridPane;
import javafx.scene.control.*;
import javafx.geometry.*;
public class ShowGridPane extends Application {
  @Override
  public void start(Stage primaryStage) {
    // Create a pane and set its properties
    GridPane pane = new GridPane();
    pane.setAlignment(Pos.CENTER);
    pane.setPadding(new Insets(11.5, 12.5, 13.5, 14.5));
    pane.setHgap(5.5);
    pane.setVgap(5.5);
    // Place nodes in the pane at positions column,row
    pane.add(new Label("First Name:"), 0, 0);
    pane.add(new TextField(), 1, 0);
    pane.add(new Label("MI:"), 0, 1);
    pane.add(new TextField(), 1, 1);
    pane.add(new Label("Last Name:"), 0, 2);
    pane.add(new TextField(), 1, 2);
    Button btAdd = new Button("Add Name");
    pane.add(btAdd, 1, 3);
    GridPane.setHalignment(btAdd, HPos.RIGHT);
    // Create a scene and place it in the stage
    Scene scene = new Scene(pane);
    primaryStage.setTitle("ShowGridPane");
    primaryStage.setScene(scene); primaryStage.show(); }
  …
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

BorderPane

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.layout.BorderPane

```
-top: ObjectProperty<Node>
-right: ObjectProperty<Node>
-bottom: ObjectProperty<Node>
-left: ObjectProperty<Node>
-center: ObjectProperty<Node>
```

```
+BorderPane()
+setAlignment(child: Node, pos: Pos)
```

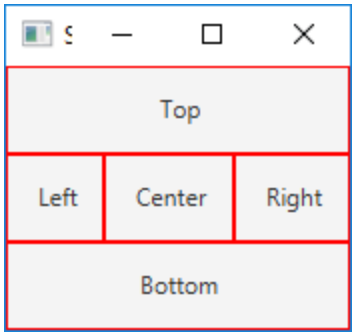
The node placed in the top region (default: `null`).
The node placed in the right region (default: `null`).
The node placed in the bottom region (default: `null`).
The node placed in the left region (default: `null`).
The node placed in the center region (default: `null`).

Creates a `BorderPane`.
Sets the alignment of the node in the `BorderPane`.

ShowBorderPane

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Label;
import javafx.geometry.Insets;
public class ShowBorderPane extends Application {
  @Override
  public void start(Stage primaryStage) {
    BorderPane pane = new BorderPane();
    pane.setTop(new CustomPane("Top"));
    pane.setRight(new CustomPane("Right"));
    pane.setBottom(new CustomPane("Bottom"));
    pane.setLeft(new CustomPane("Left"));
    pane.setCenter(new CustomPane("Center"));
    Scene scene = new Scene(pane);
    primaryStage.setTitle("ShowBorderPane"); // Set the stage title
    primaryStage.setScene(scene); primaryStage.show();
  }
  /* main method defined here */
}
class CustomPane extends StackPane {
  public CustomPane(String title) {
    getChildren().add(new Label(title));
    setStyle("-fx-border-color: red");
    setPadding(new Insets(11.5, 12.5, 13.5, 14.5));

  }}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

HBox

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.layout.HBox

```
-alignment: ObjectProperty<Pos>
-fillHeight: BooleanProperty
-spacing: DoubleProperty
```

```
+HBox()
+HBox(spacing: double)
+setMargin(node: Node, value:
   Insets): void
```

The overall alignment of the children in the box (default: `Pos.TOP_LEFT`).

Is resizable children fill the full height of the box (default: `true`).

The horizontal gap between two nodes (default: `0`).

Creates a default `HBox`.

Creates an `HBox` with the specified horizontal gap between nodes.

Sets the margin for the node in the pane.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

VBox

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.layout.VBox

```
-alignment: ObjectProperty<Pos>
-fillWidth: BooleanProperty
-spacing: DoubleProperty
```

```
+VBox()
+VBox(spacing: double)
+setMargin(node: Node, value:
   Insets): void
```

The overall alignment of the children in the box (default: `Pos.TOP_LEFT`).

Is resizable children fill the full width of the box (default: `true`).

The vertical gap between two nodes (default: `0`).

Creates a default `VBox`.

Creates a `VBox` with the specified horizontal gap between nodes.

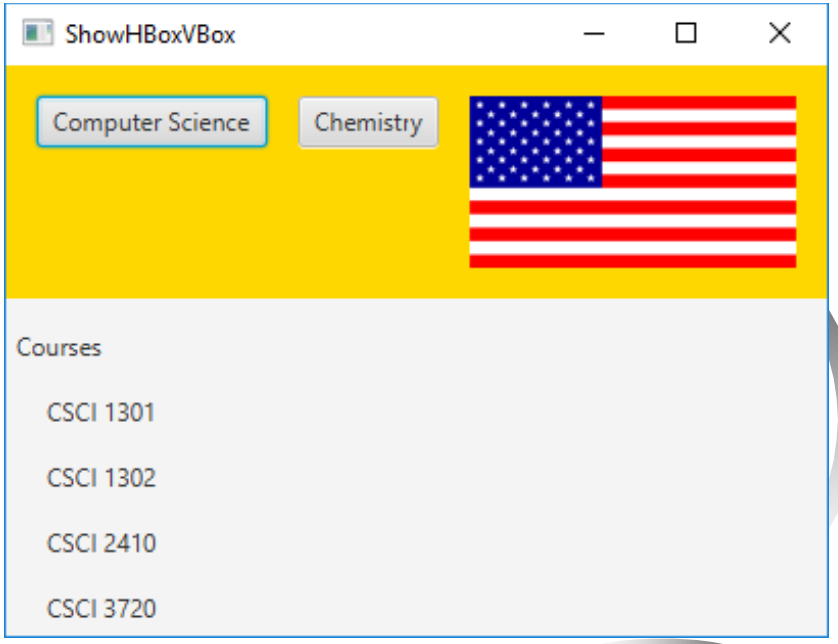
Sets the margin for the node in the pane.

ShowHBoxVBox

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;
import javafx.scene.image.*;

public class ShowHBoxVBox extends Application {
  @Override // Override the start method in the
Application
  public void start(Stage primaryStage) {
    // Create a border pane
    BorderPane pane = new BorderPane();

    // Place nodes in the pane
    pane.setTop(getHBox());
    pane.setLeft(getVBox());

    // Create a scene and place it in the stage
    Scene scene = new Scene(pane);
    primaryStage.setTitle("ShowHBoxVBox");
    primaryStage.setScene(scene);
    primaryStage.show();
  }

  private HBox getHBox() {
    HBox hBox = new HBox(15);
    hBox.setPadding(new Insets(15, 15, 15, 15));
    hBox.setStyle("-fx-background-color: gold");
    hBox.getChildren().add(new Button("Computer
Science"));
    hBox.getChildren().add(new Button("Chemistry"));
    ImageView imageView = new ImageView(new
Image("image/us.gif"));
    hBox.getChildren().add(imageView);
    return hBox;
  }
```

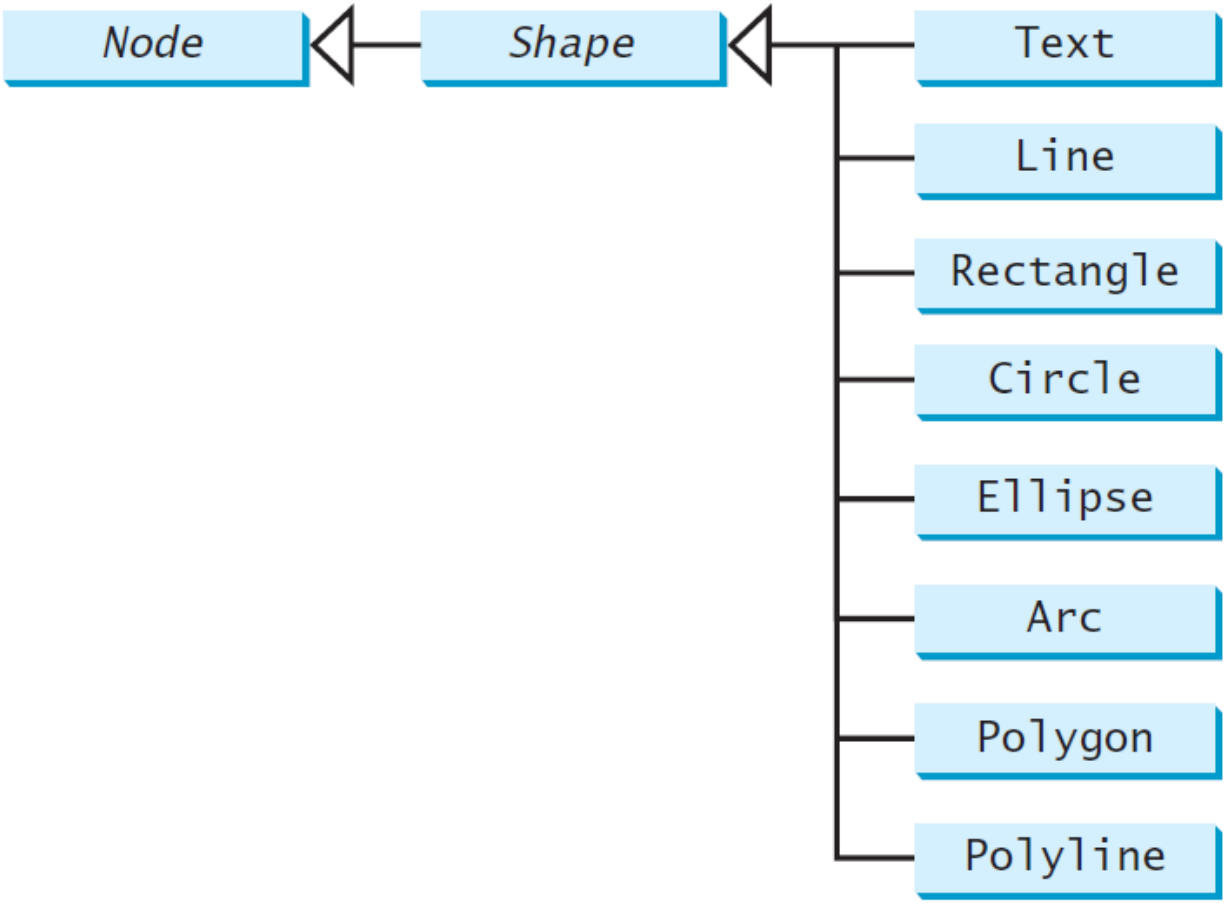
```java
 private VBox getVBox() {
    VBox vBox = new VBox(15);
    vBox.setPadding(new Insets(15, 5, 5, 5));
    vBox.getChildren().add(new Label("Courses"));

    Label[] courses = {new Label("CSCI 1301"), new
Label("CSCI 1302"), new Label("CSCI 2410"), new
Label("CSCI 3720")};

    for (Label course: courses) {
      VBox.setMargin(course, new Insets(0, 0, 0, 15));
      vBox.getChildren().add(course);
    }

    return vBox;
  }
public static void main(String[] args) {
    launch(args);
  }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Shapes

JavaFX provides many shape classes for drawing texts, lines, circles, rectangles, ellipses, arcs, polygons, and polylines.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Text

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.text.Text

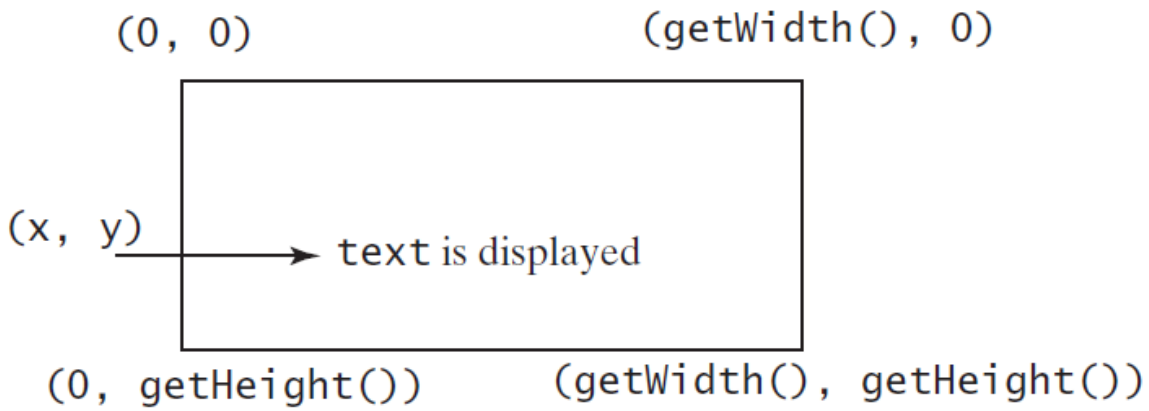
```
-text: StringProperty
-x: DoubleProperty
-y: DoubleProperty
-underline: BooleanProperty
-strikethrough: BooleanProperty
-font: ObjectProperty<Font>
```

```
+Text()
+Text(text: String)
+Text(x: double, y: double,
   text: String)
```

Defines the text to be displayed.	
Defines the x-coordinate of text (default 0).	
Defines the y-coordinate of text (default 0).	
Defines if each line has an underline below it (default `false`).	
Defines if each line has a line through it (default `false`).	
Defines the font for the text.	
Creates an empty Text.	
Creates a Text with the specified text.	
Creates a Text with the specified x-, y-coordinates and text.	

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Text Example



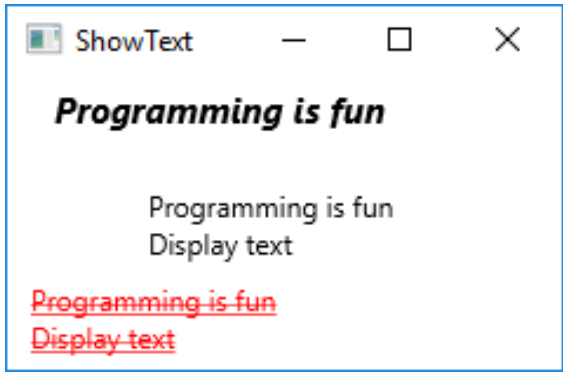
(a) Text(x, y, text)

(b) *Three* Text *objects are displayed*

ShowText

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.geometry.Insets;
import javafx.scene.text.Text;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.FontPosture;
public class ShowText extends Application {
  @Override
  public void start(Stage primaryStage) {
    Pane pane = new Pane();
    pane.setPadding(new Insets(5, 5, 5, 5));
    Text text1 = new Text(20, 20, "Programming is fun");
    text1.setFont(Font.font("Courier", FontWeight.BOLD,
      FontPosture.ITALIC, 15));
    pane.getChildren().add(text1);
    Text text2 = new Text(60, 60, "Programming is fun\nDisplay text");
    pane.getChildren().add(text2);
    Text text3 = new Text(10, 100, "Programming is fun\nDisplay
      text");
    text3.setFill(Color.RED);
    text3.setUnderline(true);
    text3.setStrikethrough(true);
    pane.getChildren().add(text3);
    Scene scene = new Scene(pane);
    primaryStage.setTitle("ShowText");
    primaryStage.setScene(scene); primaryStage.show();
  }
...
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Line

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

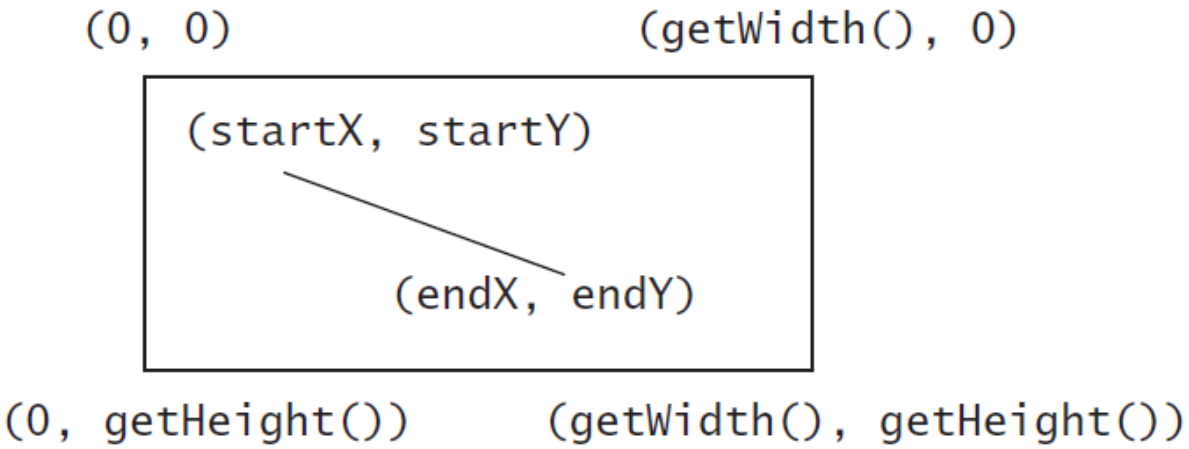
javafx.scene.shape.Line

```
-startX: DoubleProperty
-startY: DoubleProperty
-endX: DoubleProperty
-endY: DoubleProperty
```

```
+Line()
+Line(startX: double, startY:
   double, endX: double, endY:
   double)
```

The x-coordinate of the start point.
The y-coordinate of the start point.
The x-coordinate of the end point.
The y-coordinate of the end point.

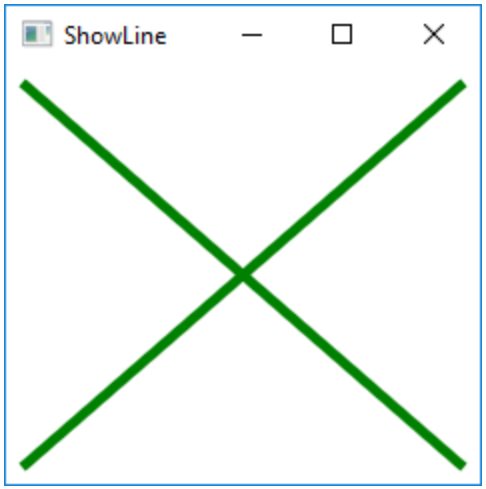
Creates an empty `Line`.
Creates a `Line` with the specified starting and ending points.



ShowLine

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.stage.Stage;
import javafx.scene.shape.Line;

public class ShowLine extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a scene and place it in the stage
    Scene scene = new Scene(new LinePane(), 200, 200);
    primaryStage.setTitle("ShowLine"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
  public static void main(String[] args) {
    launch(args);
  }
}
class LinePane extends Pane {
  public LinePane() {
    Line line1 = new Line(10, 10, 10, 10);
    line1.endXProperty().bind(widthProperty().subtract(10));
    line1.endYProperty().bind(heightProperty().subtract(10));
    line1.setStrokeWidth(5);
    line1.setStroke(Color.GREEN);
    getChildren().add(line1);

    Line line2 = new Line(10, 10, 10, 10);
    line2.startXProperty().bind(widthProperty().subtract(10));
    line2.endYProperty().bind(heightProperty().subtract(10));
    line2.setStrokeWidth(5);
    line2.setStroke(Color.GREEN);
    getChildren().add(line2);
  }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Rectangle

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.shape.Rectangle

```
-x: DoubleProperty
-y:DoubleProperty
-width: DoubleProperty
-height: DoubleProperty
-arcWidth: DoubleProperty

-arcHeight: DoubleProperty
```

```
+Rectangle()
+Rectanlge(x: double, y:
   double, width: double,
   height: double)
```

The x-coordinate of the upper-left corner of the rectangle (default 0).

The y-coordinate of the upper-left corner of the rectangle (default 0).

The width of the rectangle (default: 0).

The height of the rectangle (default: 0).

The `arcWidth` of the rectangle (default: 0). `arcWidth` is the horizontal diameter of the arcs at the corner (see Figure 14.31a).

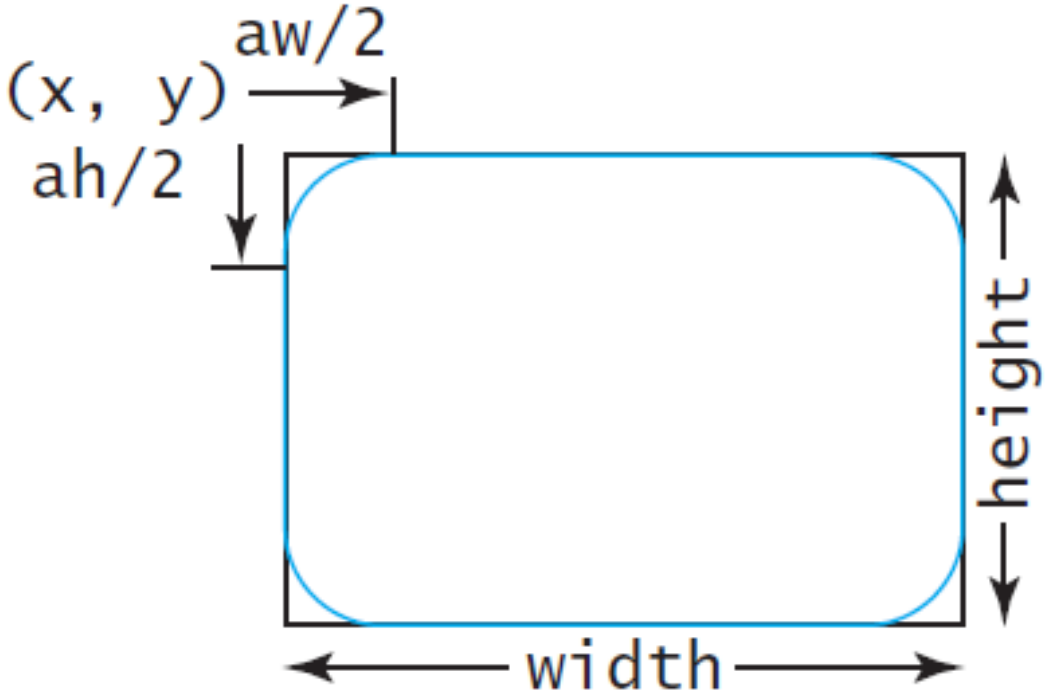
The `arcHeight` of the rectangle (default: 0). `arcHeight` is the vertical diameter of the arcs at the corner (see Figure 14.31a).

Creates an empty `Rectangle`.

Creates a `Rectangle` with the specified upper-left corner point, width, and height.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Rectangle Example

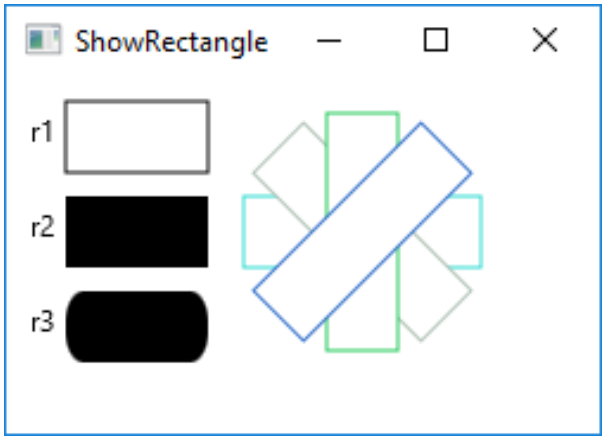


(a) Rectangle(x, y, w, h)

ShowRectangle

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
public class ShowRectangle extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a pane
    Pane pane = new Pane();

    // Create rectangles and add to pane
    Rectangle r1 = new Rectangle(25, 10, 60, 30);
    r1.setStroke(Color.BLACK);
    r1.setFill(Color.WHITE);
    pane.getChildren().add(new Text(10, 27, "r1"));
    pane.getChildren().add(r1);

    Rectangle r2 = new Rectangle(25, 50, 60, 30);
    pane.getChildren().add(new Text(10, 67, "r2"));
    pane.getChildren().add(r2);

    Rectangle r3 = new Rectangle(25, 90, 60, 30);
    r3.setArcWidth(15);
    r3.setArcHeight(25);
    pane.getChildren().add(new Text(10, 107, "r3"));
    pane.getChildren().add(r3);

    for (int i = 0; i < 4; i++) {
      Rectangle r = new Rectangle(100, 50, 100, 30);
      r.setRotate(i * 360 / 8);
      r.setStroke(Color.color(Math.random(), Math.random(), Math.random()));
      r.setFill(Color.WHITE);
      pane.getChildren().add(r);
    }

    // Create a scene and place it in the stage
    Scene scene = new Scene(pane, 250, 150);
    primaryStage.setTitle("ShowRectangle"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
  . . .
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Circle

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.shape.Circle

```
-centerX: DoubleProperty
-centerY: DoubleProperty
-radius: DoubleProperty
```

```
+Circle()
+Circle(x: double, y: double)
+Circle(x: double, y: double,
   radius: double)
```

The x-coordinate of the center of the circle (default 0).
The y-coordinate of the center of the circle (default 0).
The radius of the circle (default: 0).

Creates an empty `Circle`.
Creates a `Circle` with the specified center.
Creates a `Circle` with the specified center and radius.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Ellipse

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

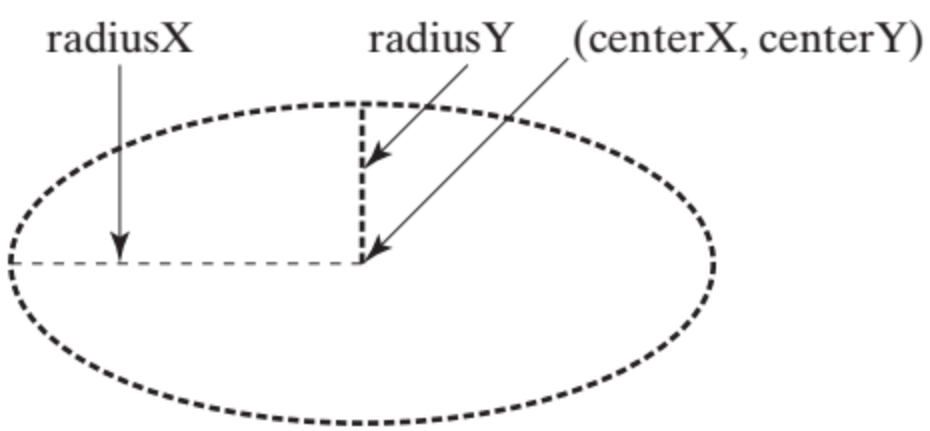
javafx.scene.shape.Ellipse

```
-centerX: DoubleProperty
-centerY: DoubleProperty
-radiusX: DoubleProperty
-radiusY: DoubleProperty
```

```
+Ellipse()
+Ellipse(x: double, y: double)
+Ellipse(x: double, y: double,
   radiusX: double, radiusY:
   double)
```

The x-coordinate of the center of the ellipse (default 0).
The y-coordinate of the center of the ellipse (default 0).
The horizontal radius of the ellipse (default: 0).
The vertical radius of the ellipse (default: 0).

Creates an empty `Ellipse`.
Creates an `Ellipse` with the specified center.
Creates an `Ellipse` with the specified center and radiuses.



ShowEllipse

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
public class ShowEllipse extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a pane
    Pane pane = new Pane();

    for (int i = 0; i < 16; i++) {
      // Create an ellipse and add it to pane
      Ellipse e1 = new Ellipse(150, 100, 100, 50);
      e1.setStroke(Color.color(Math.random(), Math.random(),
        Math.random()));
      e1.setFill(Color.WHITE);
      e1.setRotate(i * 180 / 16);
      pane.getChildren().add(e1);
    }

    // Create a scene and place it in the stage
    Scene scene = new Scene(pane, 300, 200);
    primaryStage.setTitle("ShowEllipse"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }

  /**
   * The main method is only needed for the IDE with limited
   * JavaFX support. Not needed for running from the command line.
   */
  public static void main(String[] args) {
    launch(args);
  }
}
```

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Arc

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.shape.Arc

```
-centerX: DoubleProperty
-centerY: DoubleProperty
-radiusX: DoubleProperty
-radiusY: DoubleProperty
-startAngle: DoubleProperty
-length: DoubleProperty
-type: ObjectProperty<ArcType>
```

```
+Arc()
+Arc(x: double, y: double,
   radiusX: double, radiusY:
   double, startAngle: double,
   length: double)
```

The x-coordinate of the center of the ellipse (default 0).

The y-coordinate of the center of the ellipse (default 0).

The horizontal radius of the ellipse (default: 0).

The vertical radius of the ellipse (default: 0).

The start angle of the arc in degrees.

The angular extent of the arc in degrees.

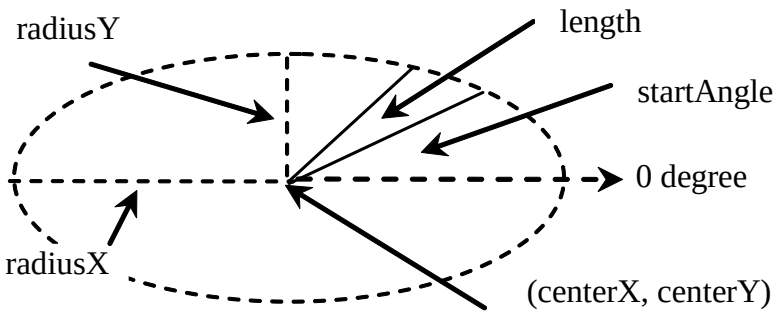
The closure type of the arc (`ArcType.OPEN`, `ArcType.CHORD`, `ArcType.ROUND`).

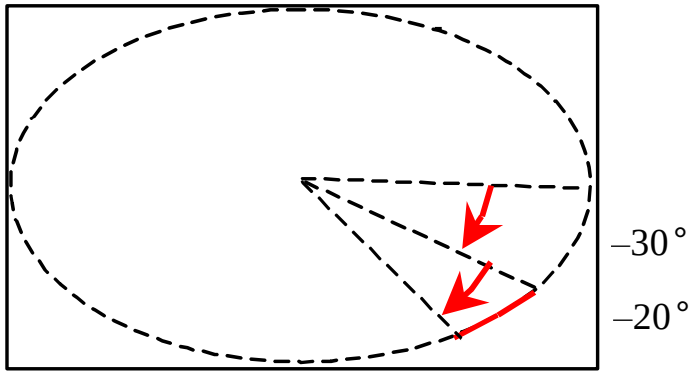
Creates an empty `Arc`.

Creates an `Arc` with the specified arguments.

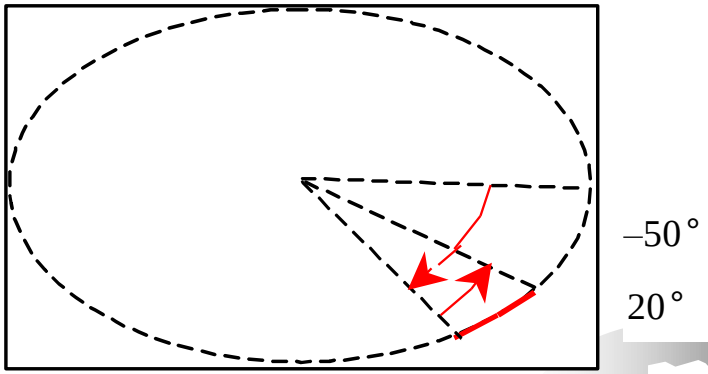
Copyright © 2022 Pearson Education Ltd. All rights reserved.

Arc Examples





(a) Negative starting angle –30° and negative spanning angle –20°

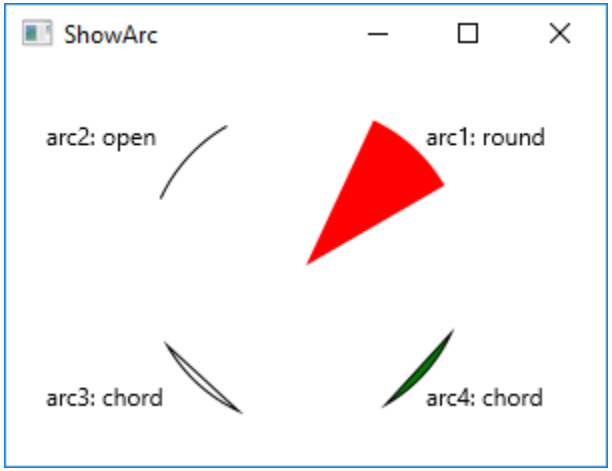


(b) Negative starting angle –50° and positive spanning angle 20°

ShowArc

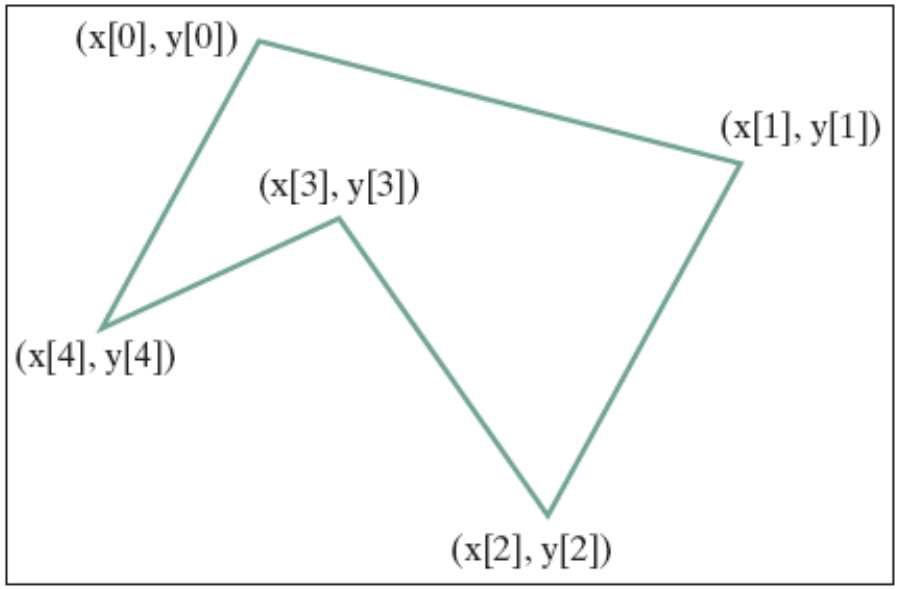
Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
public class ShowArc extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a pane
    Pane pane = new Pane();

    Arc arc1 = new Arc(150, 100, 80, 80, 30, 35); // Create an arc
    arc1.setFill(Color.RED); // Set fill color
    arc1.setType(ArcType.ROUND); // Set arc type
    pane.getChildren().add(new Text(210, 40, "arc1: round"));
    pane.getChildren().add(arc1); // Add arc to pane

    Arc arc2 = new Arc(150, 100, 80, 80, 30 + 90, 35);
    arc2.setFill(Color.WHITE);
    arc2.setType(ArcType.OPEN);
    arc2.setStroke(Color.BLACK);
    pane.getChildren().add(new Text(20, 40, "arc2: open"));
    pane.getChildren().add(arc2);

    Arc arc3 = new Arc(150, 100, 80, 80, 30 + 180, 35);
    arc3.setFill(Color.WHITE);
    arc3.setType(ArcType.CHORD);
    arc3.setStroke(Color.BLACK);
    pane.getChildren().add(new Text(20, 170, "arc3: chord"));
    pane.getChildren().add(arc3);

    Arc arc4 = new Arc(150, 100, 80, 80, 30 + 270, 35);
    arc4.setFill(Color.GREEN);
    arc4.setType(ArcType.CHORD);
    arc4.setStroke(Color.BLACK);
    pane.getChildren().add(new Text(210, 170, "arc4: chord"));
    pane.getChildren().add(arc4);

    // Create a scene and place it in the stage
    Scene scene = new Scene(pane, 300, 200);
    primaryStage.setTitle("ShowArc"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
  …
}
```

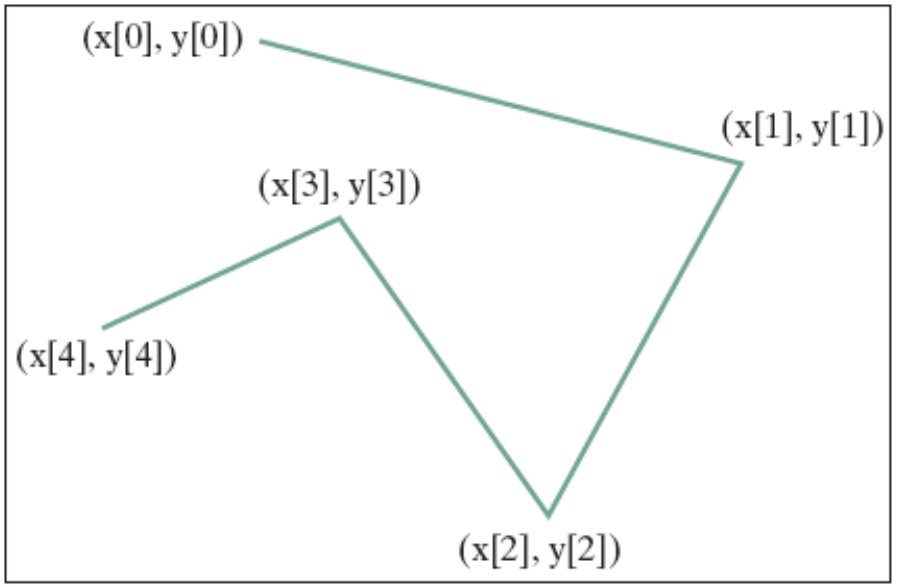


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Polygon and Polyline



(a) Polygon



(b) Polyline

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Polygon

javafx.scene.shape.Polygon
`+Polygon()`
`+Polygon(double... points)`
`+getPoints(): ObservableList<Double>`

The `getter` and `setter` methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

Creates an empty polygon.

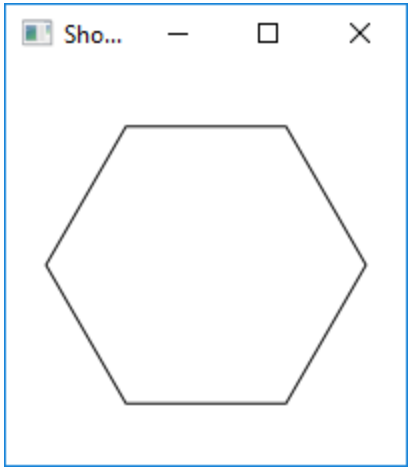
Creates a polygon with the given points.

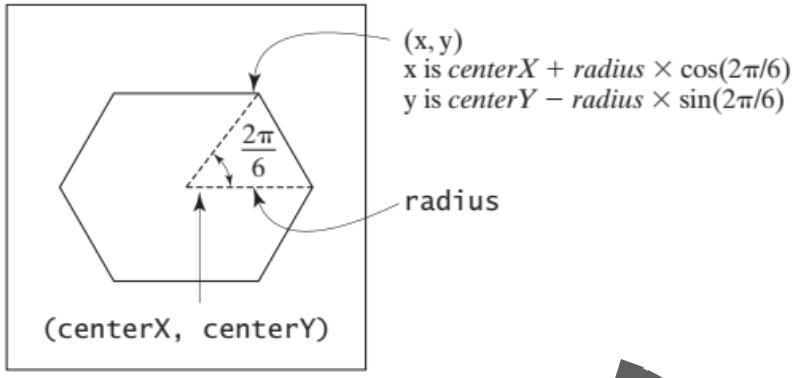
Returns a list of double values as x- and y-coordinates of the points.

ShowPolygon

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.collections.ObservableList;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.stage.Stage;
import javafx.scene.shape.Polygon;

public class ShowPolygon extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a pane, a polygon, and place polygon to pane
    Pane pane = new Pane();
    Polygon polygon = new Polygon();
    pane.getChildren().add(polygon);
    polygon.setFill(Color.WHITE);
    polygon.setStroke(Color.BLACK);
    ObservableList<Double> list = polygon.getPoints();

    final double WIDTH = 200, HEIGHT = 200;
    double centerX = WIDTH / 2, centerY = HEIGHT / 2;
    double radius = Math.min(WIDTH, HEIGHT) * 0.4;

    // Add points to the polygon list
    for (int i = 0; i < 6; i++) {
      list.add(centerX + radius * Math.cos(2 * i * Math.PI / 6));
      list.add(centerY - radius * Math.sin(2 * i * Math.PI / 6));
    }
    // Create a scene and place it in the stage
    Scene scene = new Scene(pane, WIDTH, HEIGHT);
    primaryStage.setTitle("ShowPolygon"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
  . . .
}
```

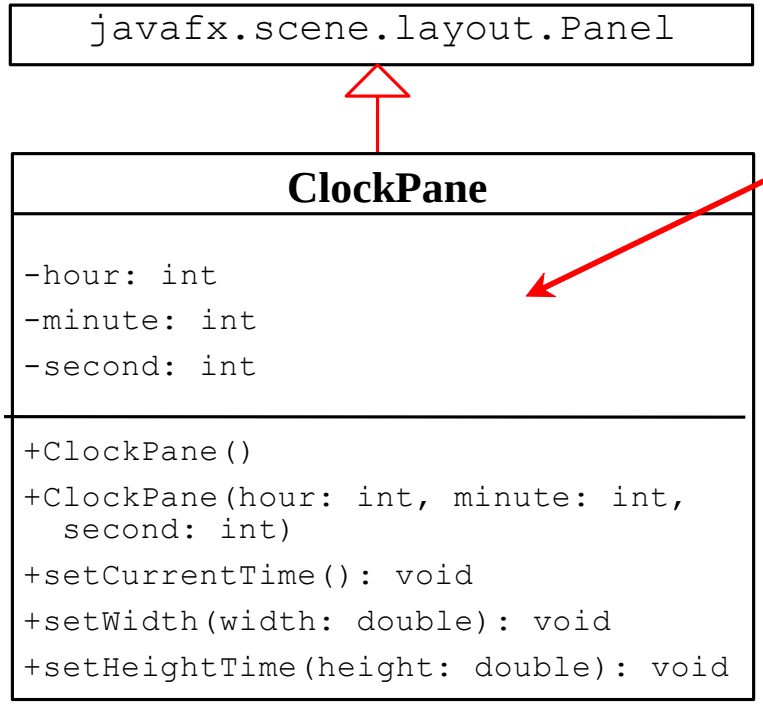




Copyright © 2022 Pearson Education Ltd. All rights reserved.

Case Study: The ClockPane Class

This case study develops a class that displays a clock on a pane.



The `getter` and `setter` methods for these data fields are provided in the class, but omitted in the UML diagram for brevity.

The hour in the clock.

The minute in the clock.

The second in the clock.

Constructs a default clock for the current time.

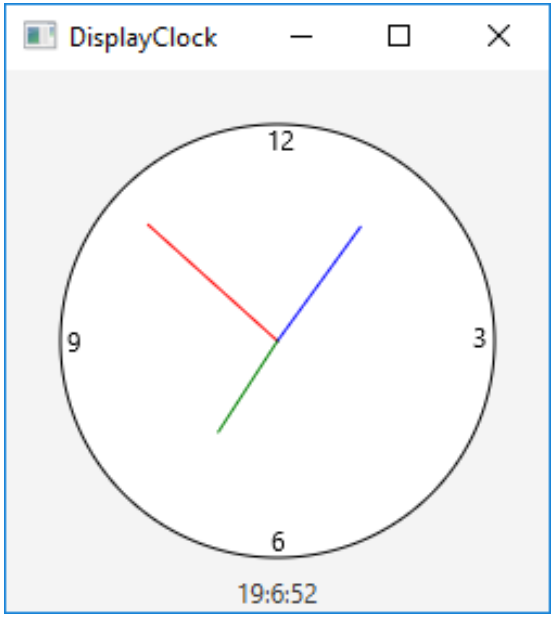
Constructs a clock with the specified time.

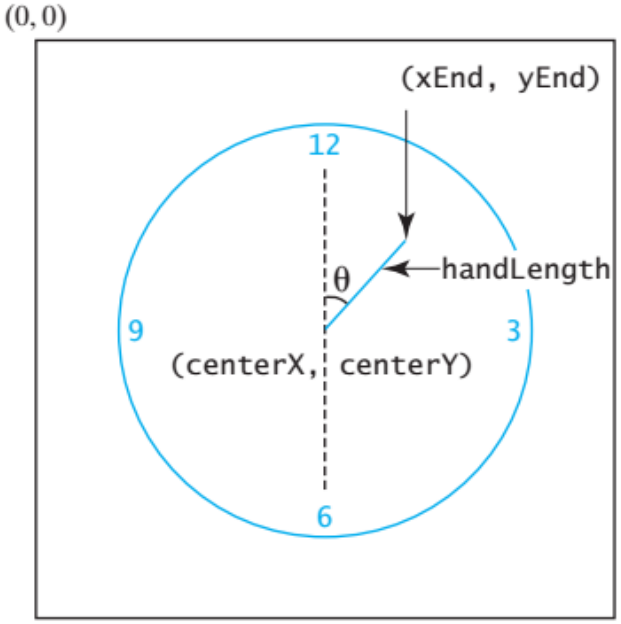
Sets hour, minute, and second for current time.
Sets clock pane's width and repaint the clock,
Sets clock pane's height and repaint the clock,

ClockPane

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Use the ClockPane Class





(b)

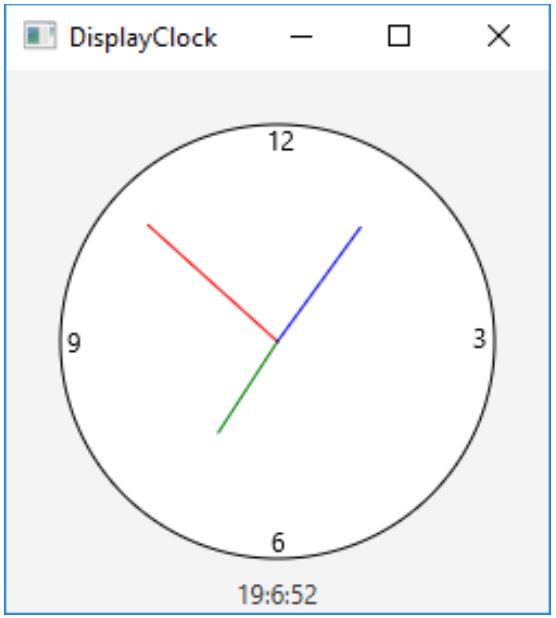
(a) The **DisplayClock** program displays a clock that shows the current time.
(b) The endpoint of a clock hand can be determined, given the spanning angle, the hand length, and the center point.

DisplayClock

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Use the ClockPane Class (cont.)

- Since there are **60 seconds** in ***one minute***, the angle for the second hand is:
  - θ = second × ($2\pi/60$)
- Since there are **60 minutes** in ***one hour***, the angle for the minute hand is:
  - θ = minute × ($2\pi/60$)
- Since one circle is divided into **12 hours**, the angle for the hour hand is:
  - θ = (hour + minute/60) × ($2\pi/12$)



Copyright © 2022 Pearson Education Ltd. All rights reserved.